That's pretty cool. Executing the tests echoes our expectations back at us, telling us that each has yet to be implemented. Fixing things is just a few keystrokes away. Let's start by meeting the first expectation—when a game starts, the score should be 0 to 0. We'll start by fleshing out the spec:

unittesting/bdd/2/ts_spec.rb

```
require_relative "tennis_scorer"

RSpec.describe TennisScorer do
  describe "basic scoring" do
    it "starts with a score of 0-0" do
      ts = TennisScorer.new
      expect(ts.score).to eq("0-0")
    end

    it "makes the score 15-0 if the server wins a point"
    it "makes the score 0-15 if the receiver wins a point"
    it "makes the score 15-15 after they both win a point"
  end
end
```

Our tests assume that we have a class TennisScorer, both in the line of code that creates an instance, but also in the top line of code RSpec.describe TennisScorer. Inside that, we have a second call to describe that groups our expectations. Our first expectation now has a code block associated with it. Inside that block, we create a TennisScorer and then use RSpec's expectation syntax to validate that the score starts out at "0-0". This particular aspect of RSpec syntax probably generates the most controversy—some people love it, others find it awkward. Either way, expect(ts.score).to eq("0-0") is equivalent to assert_equal("0-0", ts.score).

We can run our tests at this point with the same command, and we'll see the test fail because the the TennisScorer class doesn't exist.

Before we create that class and pass the test, let's take a moment to explain what RSpec is doing here. RSpec is an example of a *Domain Specific Language* (DSL), an alternate syntax built on Ruby with the goal of making it easier to express the intent of the test. Like a lot of Ruby DSL's, RSpec takes advantage of Ruby's flexibility to result in code that doesn't look exactly like regular Ruby.

When trying to understand RSpec, it's helpful to reinstate full parenthesis and implicit self message receivers as a guide to what's actually happening. Here's what that looks like for our current spec:

unittesting/bdd/2/ts_spec_paren.rb

```
require_relative "tennis_scorer"

RSpec.describe(TennisScorer) do
  self.describe("basic scoring") do
    self.it("starts with a score of 0-0") do
      ts = TennisScorer.new
      self.expect(ts.score).to(self.eq("0-0"))
    end

    self.it("makes the score 15-0 if the server wins a point")
    self.it("makes the score 0-15 if the receiver wins a point")
    self.it("makes the score 15-15 after they both win a point")
  end
end
```